

Pipeline danych ma dawac poprawny wynik nie tylko wtedy, gdy wszystko pojdzie idealnie.

Ma byc odporny na normalne problemy:

- retry,
- duplikaty,
- update,
- delete,
- spoznione dane,
- czesciowy blad zapisu,
- nagly skok metryki.

Idempotencja znaczy: powtorzenie tego samego kroku na tym samym input daje ten sam poprawny wynik.

Problem Bez Reliability

Wyobraz sobie prosty daily pipeline:

```
source orders -> INSERT INTO gold_daily_sales -> dashboard
```

Jesli job padnie i Airflow odpali retry, naiwny pipeline moze zrobic:

```
run 1: INSERT revenue 470  
run 2: INSERT revenue 470 jeszcze raz  
dashboard: revenue 940
```

SQL sie wykonal. Dashboard dziala. Wynik jest zly.

First Principles

Pipeline psuje wynik wtedy, gdy nie ma jasnych gwarancji.

Musisz wiedziec:

```
Co jest inputem?  
Co jest jednym rekordem?  
Jaki klucz identyfikuje event?  
Jaki klucz identyfikuje obiekt biznesowy?  
Który stan jest aktualny?  
Co sie stanie przy retry?  
Jaki check wykryje blad?  
Jak naprawiamy wynik?
```

Dlatego projektujesz pod:

```
raw changes -> dedupe -> current state -> validation -> publish ->  
recovery path
```

Case Z Lekcji

W labie system zrodlowy nie wysyla pelnej tabeli zamowien.

Wysyla zmiany:

```
INSERT  
UPDATE  
DELETE
```

Kazda zmiana trafia do:

```
bronze_order_changes
```

Najwazniejsza mysl:

```
Bronze changes != current orders
```

Przykład Raw Changes

Ten sam `order_id` może mieć kilka zmian:

change_id	operation	order_id	status	amount	change_timestamp
chg_001	INSERT	1001	created	120	2026-05-20 09:00
chg_002	UPDATE	1001	paid	120	2026-05-20 10:00
chg_002	UPDATE	1001	paid	120	2026-05-20 10:00

Co tu widzisz?

`order_id = 1001` ma historię zmian
`change_id = chg_002` jest duplikatem eventu

Event Key Vs Business Key

To sa dwa rozne pytania.

```
change_id -> czy to ten sam event?  
order_id  -> ktore zamowienie opisuje event?
```

Nie deduplikuj CDC od razu po `order_id`.

Dlaczego?

```
Bo UPDATE zamowienia 1001 nie jest duplikatem INSERT-a zamowienia  
1001.  
To kolejna prawdziwa zmiana tego samego obiektu biznesowego.
```

Grain W Kazdej Warstwie

Bronze changes:

```
jeden rekord = jedna surowa zmiana ze zrodla
```

Deduped changes:

```
jeden rekord = jeden unikalny change_id
```

Current state:

```
jeden rekord = najnowszy znany stan jednego order_id
```

Gold daily sales:

```
jeden rekord = jeden order_date z paid active revenue
```

Dedupe Eventow

Najpierw usuwasz duplicate event.

Minimalny wzorzec:

```
ROW_NUMBER() OVER (  
    PARTITION BY change_id  
    ORDER BY change_timestamp DESC, order_id DESC  
    ) AS row_number
```

Potem bierzesz:

```
WHERE row_number = 1
```

To chroni pipeline przed sytuacją, w której source albo retry wysłał ten sam event drugi raz.

Current State

Po dedupe nadal masz historie zmian.

Teraz wybierasz najnowszy stan zamowienia:

```
ROW_NUMBER() OVER (  
    PARTITION BY order_id  
    ORDER BY change_timestamp DESC, change_id DESC  
) AS row_number
```

Potem bierzesz:

```
WHERE row_number = 1
```

To odpowiada na pytanie:

```
Jaki jest aktualny stan order_id teraz?
```

Dlaczego Potrzebny Jest Tiebreaker

Nie wystarczy powiedzieć:

```
ORDER BY change_timestamp DESC
```

Jesli dwa eventy maja ten sam timestamp, wynik moze byc niedeterministyczny.

Dlatego dodajesz drugi warunek sortowania:

```
ORDER BY change_timestamp DESC, change_id DESC
```

Nie chodzi o to, ze `change_id DESC` zawsze jest idealne produkcyjnie.

Chodzi o zasade:

```
latest state musi miec deterministyczny porzadek
```

DELETE Nie Jest Aktywnym Zamowieniem

Jesli najnowszy event dla `order_id = 1003` to:

```
operation = DELETE
```

to ten rekord moze byc w historii zmian, ale nie powinien wejsc jako aktywne zamowienie do Silver/Gold.

Wzorzec:

```
FROM current_order_state  
WHERE operation <> 'DELETE'
```

To nie usuwa historii ze zrodla.

To tylko mowi:

```
Silver active orders nie zawiera usunietych zamowien.
```

 Refunded Nie Jest Paid Revenue

DELETE to nie jedyny edge case.

Zamowienie moze byc aktualnie:

```
created  
paid  
refunded  
deleted
```

Gold revenue w tej lekcji liczy tylko:

```
WHERE status = 'paid'
```

Dlaczego?

```
Revenue musi miec definicje biznesowa.  
Nie kazdy aktywny rekord jest przychodem.
```

Finalny Przepływ Lekcji

Docelowy pipeline logiczny:

```
bronze_order_changes  
-> deduped_order_changes  
-> current_order_state  
-> silver_orders  
-> gold_daily_sales
```

Kazdy krok ma jedną gwarancję:

```
dedupe -> duplicate event nie przejdzie dalej  
current state -> jeden order_id ma jeden aktualny stan  
silver -> DELETE nie jest aktywnym zamówieniem  
gold -> revenue liczy tylko paid orders
```

Full Load Vs Incremental Load

Full load:

przetwarzasz cały zakres danych od nowa

Incremental load:

przetwarzasz tylko nowy albo zmieniony zakres danych

Full load jest prostszy mentalnie, ale droższy przy dużych danych.

Incremental load jest wydajniejszy, ale wymaga:

- klucza,
- watermarku,
- obsługi late data,
- retry-safe write.

CDC

CDC, czyli Change Data Capture, oznacza czytanie zmian zamiast pełnej tabeli.

CDC odpowiada na pytanie:

```
Co zmieniło się w source od ostatniego razu?
```

Przykładowe operacje:

```
INSERT order 1001  
UPDATE order 1001 to paid  
DELETE order 1003
```

CDC daje historię zmian.

Nie daje automatycznie poprawnego Golda.

Watermark

Watermark to znacznik, do ktorego momentu pipeline przetworzył dane.

Przykład:

```
last_processed_change_timestamp = 2026-05-22 09:30:00
```

Pipeline może potem zapytać:

```
daj zmiany nowsze niż ostatni watermark
```

Ryzyko:

```
jeśli event przyjdzie późno ze starym order_date,  
Gold dla starego dnia może wymagać przeliczenia
```

Spoznione Dane

Spoznione dane to dane, ktore dotycza starego dnia, ale przyszly pozniej.

Przyklad z labu:

```
order_date      = 2026-05-19  
change_timestamp = 2026-05-22
```

To sa dwie rozne daty:

```
order_date      -> dzien biznesowy Gold  
change_timestamp -> kiedy pipeline zobaczyl zmiane
```

Dlatego pipeline czesto przelicza lookback window, np. ostatnie 3-7 dni.

Lookback Window

Lookback window to zakres historii, który przeliczasz przy normalnym runie.

Przykład:

```
codziennie przelicz ostatnie 3 dni biznesowe
```

Po co?

```
zeby late arriving data moglo poprawic starszy Gold
```

Koszt:

```
wieksze okno = drozsze przetwarzanie  
mniejsze okno = wieksze ryzyko pominiecia spoznionych zmian
```

To jest decyzja produktu danych, nie tylko SQL.

Retry-Safe Write

Najbardziej niebezpieczny wzorzec:

```
INSERT INTO gold_daily_sales  
SELECT ...
```

Jesli ten sam run odpali sie drugi raz, moze dopisac ten sam wynik drugi raz.

Bezpieczniejsze wzorce:

```
MERGE / UPSERT po kluczu  
DELETE controlled range + INSERT  
partition overwrite  
CREATE OR REPLACE table/view dla kontrolowanego zakresu
```

W SQLite w tej lekcji pokazujemy logike przez widoki.

W produkcji wybierasz mechanizm zapisu zgodny z platforma.

ACID

ACID to zestaw gwarancji zapisu danych.

W tej lekcji wystarczy intuicja:

```
Atomicity   -> zapis udał się cały albo wcale  
Consistency -> po zapisie dane nadal spełniają reguły  
Isolation   -> dwa równoległe zapisy nie psują sobie wyniku  
Durability  -> zatwierdzony zapis nie znika po awarii
```

Najważniejsze dla pipeline'u:

```
odbiorca nie powinien zobaczyć połowy tabeli Gold
```

ACID W Systemach Rozproszonych

W lokalnej bazie jeden silnik kontroluje zapis, blokady i commit.

W systemie rozproszonym dane mogą leżeć w wielu plikach, partycjach, workerach albo regionach.

To utrudnia pytanie:

Czy wszyscy widzą dokładnie ten sam zatwierdzony stan danych?

Dlatego formaty tabel lakehouse, np. Delta Lake, Iceberg i Hudi, dodają metadane transakcji nad plikami.

Izolacja Transakcji

Izolacja odpowiada na pytanie, co widza dwa procesy, które czytają albo zapisują dane w tym samym czasie.

Problem:

```
job kasuje partycje Gold  
job dopisuje nowe dane  
dashboard czyta w srodku operacji
```

Cel:

```
dashboard widzi poprzedni poprawny wynik albo nowy poprawny wynik
```

Nie musisz znać poziomów izolacji na pamięć. Masz rozumieć, jaki błąd izolacja zatrzymuje.

ACID Nie Zastępuje Logiki

Transakcja może ochronić zapis technicznie, ale nie zna twojej metryki.

ACID nie odpowie za ciebie:

```
Po czym deduplikować event?  
Który rekord jest aktualnym stanem order_id?  
Czy DELETE ma wejść do Gold?  
Czy refunded liczy się jako revenue?  
Czy retry podwoił wynik?
```

Dlatego reliability ma dwa poziomy:

```
storage guarantees + pipeline logic
```

Validation Checks

Pipeline bez checkow wymaga wiary.

Pipeline z checkami daje dowod.

Minimalne checki w tej lekcji:

```
duplicate change_id  
duplicate current order_id  
null keys  
negative amount  
unexpected status  
freshness / max change_timestamp  
Silver paid revenue vs Gold revenue
```

Check powinien mowic:

```
Expected: 0 rows  
Meaning: jesli sa wiersze, to ...
```

Run Metadata

Run metadata odpowiada na pytania operacyjne:

```
Który pipeline sie odpalil?  
Jaki mial run_id?  
Jakie okno danych przetwarzal?  
Czy skonczyl sie sukcesem?  
Ile rekordow weszlo i wyszlo?  
Jaki byl blad?
```

Przykładowe pola:

```
pipeline_name  
run_id  
window_start  
window_end  
status  
row_count_bronze  
row_count_silver  
row_count_gold  
error_message
```

Partial Failure

Partial failure to sytuacja, w ktorej job zrobil czesc pracy i padl.

Przyklad:

```
Bronze loaded  
Silver rebuilt  
Gold failed halfway
```

Bez run metadata nie wiesz, co zostalo zrobione.

Bez idempotencji boisz sie odpalic retry.

Poprawny cel:

```
wiem, gdzie job padl i moze bezpiecznie powtorzyc kontrolowany  
zakres
```

Runbook

Runbook to instrukcja naprawy incidentu.

Nie piszesz:

```
sprawdz dane
```

Piszesz konkretnie:

```
1. sprawdz duplicate change_id  
2. sprawdz current state per order_id  
3. porownaj Silver paid revenue z Gold  
4. sprawdz max change_timestamp  
5. przelicz zakres dat X-Y
```

Runbook ma pozwolic naprawic pipeline wtedy, gdy nie pamietasz juz wszystkich decyzji projektowych.

Incident: Revenue +40%

Jesli Gold revenue nagle skoczylo o 40%, nie zaczynasz od przepisywania SQL.

Najpierw pytasz:

```
Czy source wyslal duplikaty?  
Czy retry dopisal Gold drugi raz?  
Czy current state ma wiele rekordow per order_id?  
Czy do revenue weszly statusy inne niz paid?  
Czy late data zmienila starszy dzien?
```

To jest roznica miedzy debugowaniem losowym a debugowaniem przez failure modes.

Czego Nie Robimy W Tej Lekcji

Nie uczymy sie jeszcze pelnej produkcyjnej implementacji:

- Airflow DAGow,
- dbt incremental models,
- Delta Live Tables,
- Kafka offset management,
- wszystkich poziomow izolacji transakcji,
- SCD Type 2 end-to-end.

Uczymy sie fundamentu:

jak rozpoznać failure mode i zaprojektować gwarancje danych

Otworz I Zrob To

1. Wroc do lekcji 03, jesli nie umiesz wyjasnic Bronze/Silver/Gold.
2. Przypomnij sobie `ROW_NUMBER()` i wybieranie najnowszego rekordu per klucz biznesowy.
3. Zapisz po jednym zdaniu dla pojec z sekcji `Słownik niezawodnosci`.

Musisz Umiec Odpowiedziec Przed Labem

1. Co stanie sie, jesli ten sam batch uruchomimy dwa razy?
2. Czym rozni sie append od overwrite albo upsert?
3. Po czym poznasz duplikat eventu?
4. Czym rozni sie event key od business key?
5. Czym rozni sie surowa zmiana w Bronze od aktualnego stanu w Silver?
6. Jak pipeline powinien zareagowac na spoznione dane?
7. Co chroni ACID, a czego ACID nie wie o twojej metryce?

Fundamenty SQL

1. Jak działa `ROW NUMBER()`?
2. Jak wybrac najnowszy rekord per `order id`?
3. Czym jest klucz biznesowy?
4. Czym rozni sie event od aktualnego stanu rekordu?
5. Jak sprawdzic duplikaty po `change id`?
6. Jak porownac revenue z Silver i Gold?

To są pojęcia, do których wracasz w labie i homeworku:

- idempotencja - ten sam krok uruchomiony drugi raz daje ten sam poprawny efekt,
- retry - ponowienie kroku po błędzie,
- deduplikacja - usunięcie albo pominięcie duplikatów,
- full load - przetworzenie całego zakresu danych od nowa,
- incremental load - przetworzenie tylko nowych albo zmienionych danych,
- CDC - przechwytywanie zmian w danych źródłowych,
- watermark - znacznik, do którego momentu dane zostały przetworzone,
- late data - dane, które przyszły po oczekiwanym czasie,
- backfill - uzupełnienie albo przeliczenie danych historycznych,
- runbook - instrukcja, co zrobić przy awarii albo podejrzanym wyniku.

Wzorzec 1: Bronze To Nie Current State

Antywzorzec:

```
Bronze ma order_id, wiec traktuje to jak aktualne zamowienia.
```

Lepszy wzorzec:

```
Bronze changes -> dedupe po change_id -> latest state po order_id  
-> Silver current state
```

Wzorzec 2: Retry-Safe Load

Antywzorzec:

Kazde ponowienie robi INSERT do tabeli docelowej.

Lepszy wzorzec:

staging -> walidacja -> merge/upsert albo kontrolowany overwrite partycji

Pytanie kontrolne:

Co stanie sie, jesli ten sam input przetworze drugi raz?

Wzorzec 3: Check Przed Zaufaniem

Nie wystarczy, że SQL się wykonał.

Musisz mieć check:

```
Czy wynik jest logicznie poprawny?
```

Przykład:

```
Silver paid revenue == Gold revenue
```

Jesli check nie przechodzi, pipeline nie publikuje zaufanego Gold.

Mini-Checklist

- Co jest grain Bronze?
- Co jest grain current state?
- Po czym deduplikujesz eventy?
- Po czym wybierasz najnowszy stan?
- Jak obsługujesz DELETE?
- Jak filtrujesz paid revenue?
- Jak retry nie podwoi danych?
- Jak obsłuzysz late data?
- Co chroni ACID?
- Co zrobisz, gdy Gold revenue skoczy o 40%?

Zapamiętaj

Reliable pipeline = stable keys + deterministic state + retry-safe write + validation + recovery path

Najważniejsza odpowiedź z lekcji:

Nie ufam Gold dlatego, że job się odpalił.
Ufam Gold dlatego, że wiem, jaki failure mode obsłużyłem i jaki check to potwierdza.